rebound_rust

Provenance of the Pure-Rust Ports of REBOUND 5.1.1 and REBOUNDx 5.1.0

rustSolveIt / Windows 11 port verification record

August 27, 2026

Contents

REBOUND and REBOUNDx in Pure Rust — Complete Guide and Provenance

What this document is. Everything about two computer programs we translated: what they are, how to install and use them, exactly how they were built, and the complete evidence that the translation is correct.

Who it is for. Anyone — including someone who has never written a line of code. Technical terms are explained the first time they appear. Every command you need is printed in full, so you never have to open another document.

The one-sentence summary. We rewrote two professional astronomy programs from the C language into the Rust language, and proved the rewrite produces *exactly* the same numbers — every bit of every number — as the originals.

Which edition this is. This is the **macOS on Apple Silicon** edition of the document: every command is for the macOS Terminal, and every measured result in Part III was produced on the Apple-Silicon machine described in section 9, against the original C compiled with Apple's clang on that same machine. The port was first made and verified on Windows 11; where the Windows experience taught something worth keeping — a compiler quirk, a detective story — it is retold here and clearly labelled as the Windows story.

Part I — Understanding

1. What are these programs?

REBOUND

REBOUND answers the question: *given some objects in space, where will they be later?*

You tell it the objects — their masses, positions and velocities — and it calculates how gravity moves them. The objects can be planets orbiting a star, moons orbiting a planet, asteroids, or billions of ice chunks in Saturn's rings. It is used in hundreds of published astronomy papers.

It was written in the **C** programming language by **Hanno Rein** (University of Toronto) and collaborators. It is free and open-source.

The hard part of such a program is *accuracy over long times*. If you simulate the Solar System for a billion years, tiny errors at each step pile up. REBOUND contains about a dozen different **integrators** — different mathematical recipes for stepping forward in time — each making a different trade between speed and accuracy. Choosing the right one is most of the skill in using it.

REBOUNDx

REBOUNDx ("REBOUND eXtras") adds physics beyond plain gravity. Written by **Dan Tamayo**, Hanno Rein and collaborators, it lets you switch on effects like:

- **general relativity** — Einstein's correction to Newton's gravity, which makes Mercury's orbit slowly rotate;
- **tides** — the stretching a planet feels from its star, which slowly circularises orbits (and is why our Moon always shows us the same face);
- **spin evolution** — how a planet's rotation axis tips and drifts over time;
- **radiation pressure** — sunlight physically pushing on dust grains;
- **migration** — planets spiralling inward or outward through a gas disk.

You pick the effects you want and attach them to a REBOUND simulation.

2. What is a "port", and why do one?

A **port** is a rewrite of a program in a different language that keeps exactly what it does.

We ported both programs from **C** to **Rust**.

Why? C is fast, and REBOUND's C is well-written — but the C language itself allows a category of mistakes that are easy to make and hard to find:

- reading or writing past the end of a list (**buffer overrun**),
- using memory after it has been given back (**use-after-free**),
- forgetting to give memory back at all (**memory leak**).

These do not always crash. Sometimes they silently corrupt a number, and you publish a wrong result.

Rust refuses to compile code that could do these things. That guarantee is enforced by the compiler, not by the programmer's care. Our translation goes further and switches on `#![forbid(unsafe_code)]`, which removes even Rust's own escape hatch. So there is no place in this code where those bugs can hide.

We also used **zero third-party libraries**. The only thing our code depends on is Rust's own standard library. Nothing is downloaded when you build it.

The catch. A rewrite that changes the numbers is useless — worse than useless, because you might trust it. So the whole project rests on proving the numbers did not change. That proof is Part III of this document.

3. What "bit-for-bit identical" means

Computers store decimal numbers as 64 ones-and-zeros, in a standard format called **IEEE-754 double precision**. Those 64 ones-and-zeros are called **bits**.

When we say two results are **bit-for-bit identical**, we mean *all 64 bits are the same*, for every number. Not "the same to 10 decimal places" — identical.

Why insist on something so strict? Because these simulations are **chaotic**. A difference in the very last bit — about 1 part in 10 million billion — grows each step. After enough steps, two runs that differed by one bit look completely different. Both are still valid answers, but they are not the *same* answer.

So bit-for-bit agreement is the only test that actually proves the arithmetic is the same. Anything weaker ("agrees to 8 digits") could be hiding a real bug that has not yet had time to grow.

To test it, both programs print every number as its raw bits in **hexadecimal** (base 16, digits 0-9 and a-f). The number 1.0 prints as `3ff0000000000000`. We then compare the files character by character.

> **ULP.** You will see "ULP" below. It means *Unit in the Last Place*: the smallest difference two neighbouring representable numbers can have. A "1 ULP difference" is the smallest possible disagreement.

Part II — Using it

4. What you need installed

You need a Mac with an Apple Silicon processor (any M-series chip) and two free installs.

> ### One thing to know before you read any command in this document Every command below says **`~/work`**. That is a stand-in for *whatever folder you put this project in* — it is not a real folder on your machine and not one you have to create with that exact name. (`~` is the terminal's shorthand for your home folder, such as `/Users/sam`.) So if you cloned everything into `~/Documents/astronomy`, then wherever this document says " `cd ~/work/rebound_rust` ' you type ' `cd ~/Documents/astronomy/rebound_rust` ' Only the front part changes. Everything after ~/work/ - the folder names rebound_rust, reboundx_rust, rebound/rebound/src, porttest and so on - is real and must match exactly, because that is the layout the code expects. In this repository, ~/work is simply the repository root: cloning rustSolveIt_macos-silicon_SUNDIALS_7_8_0 gives you rebound_rust/ and reboundx_rust/ side by side already, which is exactly the arrangement every command below assumes. If you would rather not retype it every time, tell the terminal once: 'bash work=~/Documents/astronomy cd "$work/rebound_rust" "

4a. The Xcode Command Line Tools

These are Apple's free developer basics: the `clang` C compiler (needed only if you want to rebuild the C originals for Part III) and the linker that Rust uses on macOS. Open the Terminal app and type:

```
xcode-select --install
```

A dialog appears; click Install. If it says the tools are already installed, you are done. Check with:

```
clang --version
```

You should see something like `Apple clang version 21.0.0`.

4b. Rust

Go to <https://rustup.rs> and run the one-line installer it shows you (it begins `curl -proto '=https' ...`). Accept the defaults — on an Apple-Silicon Mac that installs the `aarch64-apple-darwin` toolchain, which is what we want.

Check it worked — open a **new** terminal window and type:

```
rustc --version
```

You should see something like `rustc 1.94.0`.

That is everything. There is no third install: macOS already ships `make`, `curl`, `shasum` and everything else Part III uses.

5. Five-minute quick start

Let us simulate a star with one planet.

Step 1 — make a new project.

```
cd ~/Desktop
cargo new my_first_simulation
cd my_first_simulation
```

`cargo` is Rust's build tool. `cargo new` makes a folder with a small starter program inside.

Step 2 — tell it to use our library.

Open the file `Cargo.toml` in that folder and add the last line below:

```
[package]
name = "my_first_simulation"
version = "0.1.0"
edition = "2021"

[dependencies]
rebound_rs = { path = "/Users/youruser/work/rebound_rust" }
```

(Write the full path to wherever *your* `rebound_rust` folder is — `~` does not expand inside this file, so spell out `/Users/...`.)

Step 3 — write the simulation.

Replace everything in `src/main.rs` with:

```
use rebound_rs::*;

fn main() {
    // Create an empty simulation.
    let mut sim = reb_simulation_create();
    let r = &mut sim;

    // Choose units where the gravitational constant G is 1.
    // With this choice, one unit of length is one AU (the Earth-Sun
    // distance), masses are in solar masses, and 2*pi units of time is
    // one year.
    r.G = 1.0;

    // Choose the integrator. "ias15" is the most accurate general choice.
    reb_simulation_set_integrator(r, "ias15");

    // Add the star: one solar mass, sitting at the origin, not moving.
    let mut star = reb_particle::default();
    star.m = 1.0;
    reb_simulation_add(r, star);

    // Add a planet using orbital elements instead of position/velocity:
    //   m = mass, a = size of orbit, e = how squashed the orbit is
    // e = 0 is a perfect circle; e = 0.9 is a long thin ellipse.
    reb_simulation_add_fmt(r, "m a e", &[
        reb_fmt_arg::d(1e-3),   // mass: about one Jupiter
        reb_fmt_arg::d(1.0),    // a: 1 AU
        reb_fmt_arg::d(0.1),    // e: slightly elliptical
    ]);

    // Shift everything so the centre of mass does not drift.
    reb_simulation_move_to_com(r);
```

```
    // Record the starting energy so we can check accuracy at the end.
    let energy_start = reb_simulation_energy(r);

    // Integrate forward for 100 years (remember: 2*pi = 1 year).
    reb_simulation_integrate(r, 100.0 * 2.0 * std::f64::consts::PI);

    // Print where the planet ended up.
    let p = r.particles[1];
    println!("after {:.1} years:", r.t / (2.0 * std::f64::consts::PI));
    println!("  planet position: x = {:.6}, y = {:.6}", p.x, p.y);

    // Energy should be almost perfectly conserved. This is the standard
    // way to check an N-body simulation is behaving.
    let energy_end = reb_simulation_energy(r);
    let drift = ((energy_end - energy_start) / energy_start).abs();
    println!("  relative energy drift: {:.3e}   (smaller is better)", drift);
}
```

Step 4 — run it.

```
cargo run --release
```

`-release` turns on optimisation; without it the program runs perhaps 10-50 times slower. Always use `-release` for real work.

You should see something like:

```
after 100.0 years:
  planet position: x = 0.246238, y = -0.945196
  relative energy drift: 3.331e-16
```

An energy drift of about 10^-16 is the smallest a computer can represent — the simulation is as accurate as double-precision arithmetic allows.

Congratulations, you have run an N-body simulation.

6. Cookbook: how to do the common things

All snippets assume `use rebound_rs::*;` and a simulation `r`.

Choose an integrator

```
reb_simulation_set_integrator(r, "ias15");
```

Which one should you use?

Name	Use it when	Notes
`"ias15"`	**default choice**; you want accuracy and do not know what else to pick	adaptive: chooses its own step size
`"whfast"`	long runs of a planetary system, speed matters	you must set `r.dt` yourself; very fast

Name	Use it when	Notes
`"mercurius"`	planets that occasionally have close encounters	switches to IAS15 near encounters
`"trace"`	close encounters, and you need time-reversibility	
`"saba"`	high-accuracy long-term planetary runs	family of high-order methods
`"sei"`	shearing-sheet (a patch of a planetary ring)	
`"leapfrog"`	teaching, simple problems	
`"janus"`	exactly reversible integer-based arithmetic	
`"eos"`	embedded operator splitting	
`"bs"`	you also need to evolve your own equations alongside	adaptive
`"none"`	you want particles not to move	

Set the timestep (fixed-step integrators)

```
r.dt = 0.01;
```

Rule of thumb: about 1/20th of the shortest orbital period in your simulation.

Add a particle by position and velocity

```
let mut p = reb_particle::default();
p.m = 1e-3;
p.x = 1.0;  p.y = 0.0;  p.z = 0.0;
p.vx = 0.0; p.vy = 1.0; p.vz = 0.0;
p.r = 4.6e-5;              // physical radius (needed for collisions/tides)
reb_simulation_add(r, p);
```

Add a particle by orbital elements

```
reb_simulation_add_fmt(r, "m a e inc Omega omega f", &[
    reb_fmt_arg::d(1e-3),   // mass
    reb_fmt_arg::d(5.2),    // semi-major axis (orbit size)
    reb_fmt_arg::d(0.048),  // eccentricity (0 = circle)
    reb_fmt_arg::d(0.02),   // inclination (tilt), radians
    reb_fmt_arg::d(1.75),   // longitude of ascending node, radians
    reb_fmt_arg::d(0.257),  // argument of pericentre, radians
    reb_fmt_arg::d(0.0),    // true anomaly (where on the orbit), radians
]);
```

You may give any subset. Unspecified angles default to 0. There is also a built-in Solar System:

```
reb_simulation_add_fmt(r, "solar system", &[]);        // Sun + 8 planets
reb_simulation_add_fmt(r, "outer solar system", &[]);  // Sun + Jupiter..Neptune
```

(These require `r.G = 1.0`.)

Read orbital elements back out

```
let orbit = reb_orbit_from_particle(r.G, r.particles[1], r.particles[0]);
println!("a = {}, e = {}, inc = {}", orbit.a, orbit.e, orbit.inc);
```

Integrate

```
reb_simulation_integrate(r, 1000.0);   // integrate until time t = 1000
reb_simulation_steps(r, 10_000);       // or: take exactly 10000 steps
```

Check accuracy with energy

```
let e0 = reb_simulation_energy(r);
reb_simulation_integrate(r, 1000.0);
let e1 = reb_simulation_energy(r);
println!("relative energy drift: {:e}", ((e1 - e0) / e0).abs());
```

Do something during the run (heartbeat)

```
fn my_heartbeat(r: &mut reb_simulation) {
    if reb_simulation_output_check(r, 10.0) {   // every 10 time units
        println!("t = {}", r.t);
    }
}
r.heartbeat = Some(my_heartbeat);
```

Turn on collisions

```
r.collision = REB_COLLISION::DIRECT;
r.collision_resolve = Some(reb_collision_resolve_hardsphere);
```

Particles need a physical radius `p.r` for this to do anything.

Save and reload a simulation

Files are interchangeable with C-REBOUND: a file written here loads in the C program and vice versa.

```
reb_simulation_save_to_file(r, Some("run.bin"));               // save a snapshot
reb_simulation_save_to_file_interval(r, "run.bin", 10.0);      // auto-save every 10 time units

let mut restored = reb_simulation_create_from_file("run.bin", -1).unwrap();  // -1 = latest
reb_simulation_integrate(&mut restored, 2000.0);
```

Watch it in a browser

```
reb_simulation_start_server(r, 1234);     // then open http://localhost:1234
reb_simulation_integrate(r, 1e6);
reb_simulation_stop_server(r);
```

The first call downloads `rebound.html` if it is not already in the folder.

Set options that live on the integrator

Some settings belong to a specific integrator, so you reach them like this:

```
if let reb_integrator_state::whfast(ref mut wh) = r.integrator {
    wh.corrector = 17;     // higher-order symplectic corrector
```

```
    wh.safe_mode = 0;      // faster; combines steps
}

if let reb_integrator_state::ias15(ref mut ias) = r.integrator {
    ias.epsilon = 1e-9;    // accuracy target
}
```

7. Adding extra physics with REBOUNDx

Add the second dependency to `Cargo.toml`:

```
[dependencies]
rebound_rs  = { path = "C:/work/rebound_rust" }
reboundx_rs = { path = "C:/work/reboundx_rust" }
```

The pattern is always the same:

1. `rebx_attach` — connect REBOUNDx to the simulation
2. `rebx_load_force` — pick an effect by name
3. `rebx_add_force` — switch it on
4. `rebx_set_param_*` — set the numbers the effect needs

Example: general-relativistic precession

```
use rebound_rs::*;
use reboundx_rs::*;

rebx_attach(&mut sim);
let gr = rebx_load_force(&mut sim, "gr_potential").unwrap();
rebx_add_force(&mut sim, gr);

if let Some(rebx) = rebx_extras_mut(&mut sim) {
    // the speed of light, in the same units as the simulation
    rebx_set_param_double(rebx, rebx_ap::force(gr), "c", 10065.32);
}

reb_simulation_integrate(&mut sim, 1000.0);
```

Where parameters live

In C you write `&sim->particles[0].ap` to mean "particle 0's parameter list". Here you write `rebx_ap::particle(0)`. The three possibilities:

Meaning	C	Rust
a particle's parameters	`&sim->particles[i].ap`	`rebx_ap::particle(i)`
a force's parameters	`&force->ap`	`rebx_ap::force(idx)`
an operator's parameters	`&operator->ap`	`rebx_ap::operator_(idx)`

Example: tides and spin

This is the most involved effect, and the one used in our acceptance tests. A body "has structure" (can be distorted by tides) once you give it a radius, a Love number `k2`, a moment of inertia `I` and a spin vector `Omega`.

```
rebx_attach(&mut sim);
let tides = rebx_load_force(&mut sim, "tides_spin").unwrap();
rebx_add_force(&mut sim, tides);

if let Some(rebx) = rebx_extras_mut(&mut sim) {
    // the star
    rebx_set_param_double(rebx, rebx_ap::particle(0), "k2", 0.07);
    rebx_set_param_double(rebx, rebx_ap::particle(0), "I",
                          0.07 * solar_mass * solar_rad * solar_rad);
    rebx_set_param_vec3d (rebx, rebx_ap::particle(0), "Omega",
                          reb_vec3d { x: 0., y: 0., z: solar_spin });
    rebx_set_param_double(rebx, rebx_ap::particle(0), "tau", solar_tau);
}
```

Reading a parameter back

```
if let Some(rebx) = rebx_extras_ref(&sim) {
    if let Some(omega) = rebx_get_param_vec3d(rebx, rebx_ap::particle(1), "Omega") {
        println!("spin = ({}, {}, {})", omega.x, omega.y, omega.z);
    }
}
```

A getter returns `None` in exactly the cases where the C returns a null pointer — i.e. the parameter was never set.

The available effects

Kind	Names you can pass to `rebx_load_force` / `rebx_load_operator`
Relativity	`gr`, `gr_full`, `gr_potential`, `lense_thirring`
Tides & spin	`tides_spin`, `tides_constant_time_lag`, `tides_dynamical`
Migration & disks	`modify_orbits_forces`, `modify_orbits_direct`, `type_I_migration`, `exponential_migration`, `gas_damping_timescale`, `gas_dynamical_friction`
Radiation	`radiation_forces`, `yarkovsky_effect`
Gravity shape	`gravitational_harmonics`, `central_force`
Other	`stochastic_forces`, `modify_mass`, `track_min_distance`, `integrate_force`

Each effect's required parameters are documented as Rust doc comments at the top of its module, carried over from the C source. To read them:

```
cd ~/work/reboundx_rust
cargo doc --open
```

8. Every build command, in one place

From `~/work/rebound_rust`:

```
cargo build --release
cargo test  --release
cargo clippy --release --all-targets
cargo doc   --no-deps --open

cargo build --release --example shearing_sheet
cargo build --release --example shearing_sheet_test
cargo build --release --example integrators_test
cargo build --release --example libm_diff
cargo build --release --example bs_pow_diff
cargo build --release --example derivatives_test
cargo build --release --example frequency_test
cargo build --release --example archive_test
cargo build --release --example server_test
cargo build --release --example addfmt_test
```

From `~/work/reboundx_rust`:

```
cargo build --release
cargo test  --release
cargo clippy --release --all-targets
cargo doc   --no-deps --open

cargo build --release --example tides_spin_pseudo
cargo build --release --example tides_spin_kozai
cargo build --release --example tides_spin_migration
```

To run any example:

```
cargo run --release --example <name>
cargo run --release --example integrators_test -- whfast 2 500     # with arguments
```

Every one of these completes with **zero warnings**. Nothing is downloaded: both crates are `std`-only, so they build with no network connection.

Part III — Provenance: how it was made and checked

9. The machine and the tools

Everything below was done on one computer, with no Linux, no virtual machine, no GCC and no Microsoft compiler involved at any point.

Component	Value
Machine	Apple **M5 Max**, 128 GB RAM, Apple Silicon (arm64)
Operating system	macOS Tahoe 26 (26.6.1, build 25G76)
C compiler	Apple clang 21.0.0 (clang-2100.1.1.101), from the Xcode Command Line Tools
Archiver / linker	Apple `ar` and `ld`, from the same tools
Rust	`rustc 1.94.0`, `cargo 1.94.0`, target `aarch64-apple-darwin`
REBOUND source	github.com/hannorein/rebound, **5.1.1**, commit `dad5f97806ecbb408dcaff728851c64e67f9f6eb`
REBOUNDx source	github.com/dtamayo/reboundx, **5.1.0**

The port itself was first produced and verified on a Windows 11 machine (MSVC `cl` 19.51, rustc 1.91.1, `x86_64-pc-windows-msvc`); this edition's results supersede nothing from that record — they are the *same experiments re-run on this Mac*, against a C reference built with clang on this Mac.

The 3-D viewer (for completeness)

REBOUND has a native OpenGL 3-D viewer (built on a library called GLFW), and unlike on Windows, REBOUND's build system does support it on macOS. The C **reference builds used for verification here exclude it anyway** (they are compiled without the OpenGL flag), for the same reason the Windows reference did: the viewer draws pictures and computes nothing, the port's browser-based web server is the visualisation path that *is* translated, and keeping the two reference builds identical in what they contain keeps the comparison honest.

10. Building the C originals (the reference)

To prove our Rust gives the same answers, we first need the originals to compare against.

10a. REBOUND

```
cd ~/work
git clone https://github.com/hannorein/rebound.git rebound/rebound
cd rebound/rebound
git checkout dad5f97806ecbb408dcaff728851c64e67f9f6eb
```

Compile all 31 C files and collect them into a static library (a `.a` file — a bundle of compiled code that other programs can link against):

```
cd ~/work/rebound/rebound/src
clang -c -DBUILDINGLIBREBOUND -D_GNU_SOURCE -DSERVER \
```

```
      -DGITHASH=dad5f97806ecbb408dcaff728851c64e67f9f6eb \
      -O2 -ffp-contract=off *.c
ar rcs librebound_static.a *.o
```

Two of those flags decide whether a bit-exact comparison is even possible, so they are worth understanding:

- `-O2` optimises for speed, in the same family as the `/Ox` the Windows reference used.
- `-ffp-contract=off` forbids the compiler from **fusing** a multiply and an add (`a*b + c`) into one combined machine instruction. The fused form rounds once instead of twice, so it produces a *slightly different* — equally valid — answer. Apple-Silicon processors love this fusion and clang applies it by default, but the Rust compiler never does it behind your back, so the C must be told not to either. It is the clang equivalent of the `/fp:precise` the Windows reference used.

10b. REBOUNDx

```
cd ~/work
git clone https://github.com/dtamayo/reboundx.git reboundx

cd ~/work/reboundx/src
clang -c -I../../rebound/rebound/src -I. -D_GNU_SOURCE -DLIBREBOUNDX \
      -O2 -ffp-contract=off *.c
ar rcs libreboundx.a *.o
```

All 33 files compile, unmodified. That sentence is shorter than its Windows counterpart for a reason worth recording: two REBOUNDx files (`gr_full.c` and `interpolation.c`) use **C99 variable-length arrays** — arrays whose size is decided while the program runs — which Microsoft's C compiler has never supported, so the Windows reference build needed patched copies (kept in `reboundx_rust/porttest/msvc_shim/`, changing only where the memory comes from, never any arithmetic). Apple's clang supports variable-length arrays natively, so on macOS **no shim is needed and the upstream source compiles as-is**. The Rust port never had the problem on any platform: Rust's `Vec` is a growable array.

10c. The macOS `rand_r` shim — the one thing this platform *does* need

The Windows edition of this document recorded, almost in passing, that REBOUND vendors glibc's `rand_r` random-number generator directly in `rebound.c` — "that is why random initial conditions are identical across platforms". On macOS that sentence turns out to have a catch, and it cost us a morning, so here is the full story.

`rand_r` is the C library's simple repeatable random-number generator: give it the same starting **seed** and it hands back the same sequence of numbers forever. REBOUND uses it for random initial conditions (for example, scattering 1,482 ring particles in the shearing sheet from seed 42). But "the C library" is a different program on every operating system. GNU/Linux's (called **glibc**) mixes three multiply-add rounds per number; Apple's mixes differently. Same seed, different stream.

REBOUND's authors knew this, which is exactly why `rebound.c` carries its own copy of the glibc algorithm — but that copy is guarded by `#ifdef _WIN32`: it is compiled **only on Windows**, because Windows has no `rand_r` at all. On macOS the guard is false, the C reference quietly calls Apple's `rand_r`, and the very first random number differs from the one every recorded reference run used. Our

first shearing-sheet comparison placed **1,441** particles instead of 1,482 and disagreed from particle 1 — not because any physics differed, but because the random scattering did.

The fix follows the same never-touch-upstream rule as the Windows VLA shim. `rebound_rust/porttest/macos_shim` contains the identical glibc algorithm (byte for byte the same arithmetic as the `#ifdef _WIN32` block in `rebound.c`), compiled as its own object file:

```
cd ~/work/rebound_rust/porttest
clang -c -O2 -ffp-contract=off macos_shim/rand_r_glibc.c -o macos_shim/rand_r_glibc.o
```

Linking that object into a harness makes the linker resolve REBOUND's `rand_r` calls there instead of in Apple's C library, so the C reference produces the same random stream as glibc-Linux and Windows — the stream the Rust port implements on every platform (which is why the Rust side needs no shim anywhere). With the shim linked, the shearing sheet places its 1,482 particles and, as section 15.2 records, matches the Rust byte for byte.

11. How the translation was done

The rules, applied to every file in both projects:

1. **Fidelity first.** Control flow, constants, tolerances and arithmetic *order* match the C expression by expression. Floating-point addition is not associative: `(a+b)+c` and `a+(b+c)` can give different answers, so the C's exact bracketing is transcribed even where it looks redundant.
2. **Zero `unsafe`, zero dependencies, zero warnings.** Enforced by `#![forbid(unsafe_code)]` and `#![deny(warnings)]` at the top of each crate.
3. **C names are kept.** `reb_simulation_create`, `rebx_add_force`, `reb_particle` and so on, so C code and documentation read across directly.
4. **Missing symbols are reported, never invented.** If something could not be translated it is listed in section 18, not silently faked.

The one structural problem, and its solution

C code passes raw pointers around. Safe Rust does not allow two pieces of code to hold a mutable pointer to the same thing at once. REBOUND does exactly that: a step function receives both the simulation and the integrator's own state, which *lives inside* the simulation.

The consistent solution, used everywhere: **take the state out, use it, put it back.**

```
let mut state = match std::mem::replace(&mut r.integrator, reb_integrator_state::none) {
    reb_integrator_state::whfast(s) => s,
    other => { r.integrator = other; return; }
};
// ... now both ‘r‘ and ‘state‘ can be used freely ...
r.integrator = reb_integrator_state::whfast(state);
```

This is exactly equivalent to what the C does, and it is checked by the compiler. The same pattern carries REBOUNDx's state through `reb_simulation::extras`.

12. File-by-file accounting: REBOUND

All 31 C translation units, and what happened to each. **29 Rust modules, 19,075 lines.**

C file	Rust module	Status
`rebound.c`	`tools.rs`, `server.rs`, `lib.rs`	Ported: messages, `reb_exit`, `reb_strcmp_ignore_whitespace`, `reb_check_fp_contract`, favicon data, version/githash. Not applicable in Rust: `malloc` wrappers, SIGINT handler, custom-integrator registry, AVX-512 detection, OpenMP setter.
`simulation.c`	`simulation.rs`	Ported + verified
`particle.c`	`particle.rs`	Ported + verified
`tools.c`	`tools.rs`	Ported + verified (RNG family, energies, centre-of-mass, orbit conversions incl. Pal coordinates, `add_fmt` + Solar System data, MEGNO)
`boundary.c`	`boundary.rs`	Ported + verified (incl. shear ghost boxes)
`tree.c`	`tree.rs`	Ported + verified (octree as an index arena)
`gravity.c`	`gravity.rs`	Ported + verified (basic, compensated, tree, jacobi, variational)
`collision.c`	`collision.rs`	Ported + verified (all searches and resolvers)
`output.c`	`output.rs`	Ported (screenshot excluded with the display subsystem)
`transformations.c`	`transformations.rs`	Ported + verified
`rotations.c`	`rotations.rs`	Ported + verified
`derivatives.c`	`derivatives.rs`	Ported + verified — **all 65** functions
`frequency_analysis.c`	`frequency_analysis.rs`	Ported + verified (MFT / FMFT / FMFT2)
`integrator_none/sei/leapfrog.c`	`integrator/{none,sei,leapfrog}.rs`	Ported + verified
`integrator_mercurius.c`	`integrator_mercurius.rs`	Ported + verified (IAS15 encounter machinery, hooks)
`integrator_bs.c`	`integrator_bs.rs`	Ported + verified (full `reb_ode` framework)
`integrator_trace.c`	`integrator_trace.rs`	Ported + verified (reversible checks, BS/IAS15 encounter and pericentre paths)
`integrator_whfast512.c`	`integrator_whfast512.rs`	**Stubbed on Windows and on Apple Silicon** — see below
`binarydata.c`	`binarydata.rs`	Ported + verified (same byte format)
`simulationarchive.c`	`simulationarchive.rs`	Ported + verified
`server.c`	`server.rs`	Ported + verified (HTTP endpoints, base64, threading adapted)
`fmemopen.c`	—	Not applicable: replaced by `std::io::Cursor`
`display.c`, `glad.c`	—	Excluded: OpenGL (both reference C builds exclude it; section 9)
`communication_mpi.c`	—	Excluded: MPI (in neither reference C build)

About WHFast512. Its fast core is hand-written AVX-512 assembly — machine instructions that exist only on certain Intel/AMD (x86) processors. The C guards it with an architecture check, so the full path compiles only on 64-bit x86 with GCC or Clang. Under MSVC on Windows *and* under clang on Apple Silicon (which is arm64, not x86), the C compiles the same `#else // Not 64 bit, Windows + cl` branch, which contains only stubs that report "AVX512 is not supported on your platform." Our Rust reproduces exactly that reference behaviour — so on both platforms, C and Rust integrate nothing, identically.

13. File-by-file accounting: REBOUNDx

All 33 C translation units (8,452 lines of C, including headers) become 34 Rust modules (8,798 lines). Every `.c` file is accounted for below.

Core machinery

C file	Lines	Rust module	Lines	Status
`core.c`	1186	`core.rs`	1494	Ported. All **107** default parameter registrations, in the same order with the same types (verified by direct diff against the C). `rebx_load_force`/`rebx_load_operator` carry the complete name → function tables.
`rebxtools.c`	291	`rebxtools.rs`	475	Ported (com/jacobi helpers, `rebx_tools_spin_angular_momentum`, `rebx_simulation_irotate`).
`linkedlist.c`	106	—	—	**Not applicable.** These are the linked-list helpers (`rebx_add_node`, `rebx_remove_node`, `rebx_len`). In Rust the lists are `Vec`s, so the helpers become ordinary vector operations. The C's *prepend* order is preserved by inserting at index 0 — see deviation 8 in section 14.
`output.c`	293	`output.rs`	640	Binary serialization. Verified against the real C library in both directions — see §15.10.
`input.c`	732	`input.rs`	1097	Binary deserialization. Reads files written by the C, and vice versa.

Forces

C file	Lines	Rust	Lines
`tides_spin.c`	418	`tides_spin.rs`	643
`gr_full.c`	393	`gr_full.rs`	459
`tides_dynamical.c`	346	`tides_dynamical.rs`	409
`gravitational_harmonics.c`	318	`gravitational_harmonics.rs`	454
`stochastic_forces.c`	252	`stochastic_forces.rs`	385
`gr.c`	250	`gr.rs`	276
`yarkovsky_effect.c`	248	`yarkovsky_effect.rs`	418
`tides_constant_time_lag.c`	228	`tides_constant_time_lag.rs`	262
`type_I_migration.c`	214	`type_I_migration.rs`	235
`gas_dynamical_friction.c`	176	`gas_dynamical_friction.rs`	205
`central_force.c`	152	`central_force.rs`	178
`gas_damping_timescale.c`	141	`gas_damping_timescale.rs`	181
`modify_orbits_forces.c`	139	`modify_orbits_forces.rs`	178
`radiation_forces.c`	126	`radiation_forces.rs`	156
`gr_potential.c`	120	`gr_potential.rs`	131
`lense_thirring.c`	119	`lense_thirring.rs`	129
`exponential_migration.c`	112	`exponential_migration.rs`	134

C file	Lines	Rust	Lines
inner_disk_edge.c	77	inner_disk_edge.rs	56

Operators, steppers and REBOUNDx's own integrators

C file	Lines	Rust	Lines
interpolation.c	180	interpolation.rs	263
modify_orbits_direct.c	144	modify_orbits_direct.rs	165
steppers.c	130	steppers.rs	180
integrator_implicit_midpoint.c	124	integrator_implicit_midpoint.rs	210
track_min_distance.c	98	track_min_distance.rs	112
integrator_rk4.c	90	integrator_rk4.rs	243
integrate_force.c	78	integrate_force.rs	102
modify_mass.c	74	modify_mass.rs	68
integrator_rk2.c	66	integrator_rk2.rs	154
integrator_euler.c	41	integrator_euler.rs	44

Plus `types.rs` (270 lines), which has no single C counterpart: it holds the structures from `reboundx.h` together with the three substitutions that replace the C's pointer graph (documented at the top of that file).

14. Deviations from the C, and why each is safe

Every difference is mechanical — a consequence of Rust's ownership rules — and none changes a computed number.

1. **Ownership instead of pointers.** `r->particles` (a `malloc`'d block) becomes a `Vec<reb_particle>`. The octree's individually allocated cells become an index arena, rebuilt exactly as the C rebuilds cells. Particle names, which the C interns as `char*`, become an index into a name list. The `ap` and `sim` back-pointers inside `reb_particle` are dropped — functions that used them take the simulation explicitly.

2. **Integrator state.** The C's `void* state` behind a function-pointer table becomes an `enum` with one variant per integrator, taken out and put back around each step (section 11).

3. **`r->map` aliasing.** MERCURIUS and TRACE make `r->map` point at their own `encounter_map` — one array with two names. The Rust *moves* the vector in and out instead. Same contents at every observable point.

4. **Binary file pointers.** A `reb_particle` is written in its exact 112-byte C memory layout. The C stores real heap addresses in the `name`/`ap`/`sim` slots; those addresses are only ever compared for equality against addresses stored beside the name list. Rust writes 0 for `ap`/`sim` and a synthetic id for `name`, reproducing that protocol exactly — which is why archive files are interchangeable in both directions (verified, §15.5).

5. **Server threading.** The C server thread dereferences the simulation directly under a mutex. Safe Rust cannot share `&mut` across threads, so the same handshake is expressed with a shared

snapshot and key queue, serviced by the integration loop at exactly the points where the C locks and unlocks. The HTTP behaviour is unchanged (verified, §15.6).

6. **Variadic arguments.** `reb_simulation_add_fmt(r, fmt, ...)` takes its values as an ordered `&[reb_fmt_arg]` slice, consumed token by token like `va_arg`.

7. **REBOUNDx parameters.** The C stores `void* value` plus a type tag and casts on read. Rust fuses the two into one `enum`, so a wrong-type read is impossible instead of undefined. For correct programs the behaviour is identical; for incorrect ones Rust reports rather than corrupts.

8. **REBOUNDx list order.** The C's `rebx_add_node` *prepends* to its linked lists, so a list iterates in reverse insertion order — and that order decides the order accelerations are summed, which changes floating-point results. The Rust lists are `Vec`s whose **index 0 is the head**, and the add helper inserts at 0. Iteration order is identical, element for element.

9. **Removed platform branches.** OpenMP `#pragma` blocks, `reb_sigint` polling and MPI branches are omitted — none of them is compiled in the C reference build either.

10. **Uninitialised C memory.** Where C leaves struct members uninitialised (e.g. `reb_orbit_nan`), Rust zero-initialises before setting the same members.

11. **A C quirk preserved in spirit.** `rebx_additional_forces` declares `const double N = sim->N;` — a *double* — then passes it to a function taking an `int`. For any realistic particle count this conversion is exact, so the Rust uses an integer directly.

12. **The git-hash stamp in REBOUNDx binary files.** The header of a REBOUNDx binary file carries a 26-byte field recording which source revision the library was compiled from. The C fills it in from `git` at compile time via its makefile, and writes the literal `notavailable00000000000000` when git information is not available — which is the case for the reference build used here. The Rust crate is built by `cargo` and has no such compile-time git step, so it writes 26 zero bytes. This is the **only** difference between a C-written and a Rust-written binary file (26 bytes out of the 10,784 measured in the round-trip test on this machine), it contains no simulation data, and neither library reads it back for any purpose. Both libraries read each other's files correctly regardless (verified, §15.9).

15. The complete verification record

The method throughout: run the identical experiment in the clang-compiled C and in Rust, dump every value as raw IEEE-754 bits, and compare byte for byte. A run passes only if **every bit of every value** matches. Every result in this section was measured on the Apple-Silicon machine of section 9, on 2026-08-27.

The record at a glance

What was checked	Scale	Result
Maths library agreement (§15.0)	200,000 samples × 21 functions	**21 of 21 exact — `pow` included** (§16)
Integrator matrix (§15.1)	63 configurations × 500 steps	**63/63 bit-identical**

What was checked	Scale	Result
Shearing sheet (§15.2)	1,482 particles, 400 steps	**byte-identical, SHA-256 `418c864d...`**
Orbital derivatives (§15.3)	65 functions	**130/130 outputs bit-identical**
Frequency analysis (§15.4)	MFT, FMFT, FMFT2	**bit-identical**
Simulationarchive (§15.5)	C→Rust and Rust→C continuations	**bit-identical, all six directions**
Web server (§15.6)	blob served by Rust, read by C	**bit-identical state**
`add_fmt` and datasets (§15.7)	all format tokens	**bit-identical**
REBOUNDx `tides_spin` (§15.8)	3 examples × short and long runs	**6/6 bit-identical**
REBOUNDx binary files (§15.9)	round trip, both directions	**10,758 of 10,784 bytes identical; only the git-hash stamp differs**
Automated test suite — REBOUND	394 tests	**394 pass, 0 fail**
Automated test suite — REBOUNDx	137 tests	**137 pass, 0 fail**
Compiler and clippy warnings (§17)	both crates, all targets	**zero**

The test suites found three genuine translation defects during the original Windows port, which are written up honestly in §15.10 rather than quietly fixed.

To run both suites yourself:

```
cd ~/work/rebound_rust
cargo test --release
```

```
cd ~/work/reboundx_rust
cargo test --release
```

15.0 The foundation: which maths functions agree?

Before anything else we established what the two languages' maths libraries do, using a differential harness of 200,000 samples per function:

```
cd ~/work/rebound_rust
cargo build --release --example libm_diff
cd porttest
clang -O2 -ffp-contract=off libm_diff.c -lm -o libm_diff
./libm_diff
../target/release/examples/libm_diff
cmp libm_c.txt libm_rust.txt && echo "bit-identical"
```

Result: on macOS/Apple Silicon, the C and the Rust are **bit-identical for every function tested**

— **`pow` included**: `sin`, `cos`, `tan`, `atan2`, `sqrt`, `fmod`, `exp`, `log`, `cbrt` and `pow`, over all 200,000 samples each. The reason is simple: on macOS, both clang-compiled C *and* Rust resolve every one of these calls to Apple's system maths library, so there are not two implementations to disagree. (On Windows, where Rust ships its own `pow`, that one function differed from Microsoft's — the story section 16 preserves.)

This is what makes true bit-identity testing possible at all — and on this platform it holds with no exceptions.

15.1 The integrator matrix — 63 configurations

A fixed three-body problem (star of mass 1; planet of mass 10^-3 at x = 1.6 with vy = 0.5; moon of mass 10^-7 at x = 1.7, vy = 0.6, z = 0.01, vz = 0.001), G = 1, dt = 0.01, 500 steps, final state dumped as raw bits.

Build the C harness once (the shim object comes from section 10c):

```
cd ~/work/rebound_rust/porttest
clang -I../../rebound/rebound/src -D_GNU_SOURCE -O2 -ffp-contract=off \
      integrators_test.c macos_shim/rand_r_glibc.o \
      ../../rebound/rebound/src/librebound_static.a -lm -o integrators_test
```

To check a single configuration, give the harness a configuration name, a leapfrog order (ignored by the others) and a step count, then compare:

```
cd ~/work/rebound_rust/porttest
./integrators_test whfast 2 500
../target/release/examples/integrators_test whfast 2 500
cmp state_c_final.txt state_rust_final.txt && echo identical
```

`cmp` staying silent (and `identical` printing) means that configuration matched.

To check **all 63 in one go**, run the sweep script that ships in `porttest/`. It knows every configuration name, runs each one in both languages, compares the dumps and prints a tally:

```
cd ~/work/rebound_rust/porttest
bash run_integrator_matrix.sh 500
```

(`run_integrator_matrix.ps1`, beside it, is the same sweep for Windows.) It takes a few minutes and ends with:

```
63 of 63 configurations bit-identical.
ALL CONFIGURATIONS BIT-IDENTICAL
```

Result: 63 of 63 configurations bit-identical — and the sweep was run twice, before and after the `rand_r` shim of section 10c was introduced, with the same verdict both times (the matrix uses fixed starting conditions, so it never touches the random generator; that is exactly why the shim could not affect it).

One thing to know before you run it: **every harness in `porttest/` writes its results to the same two filenames**, `state_c_final.txt` and `state_rust_final.txt`. That is deliberate — it keeps the comparison command identical for every test — but it means each run overwrites the last one's dumps, and the sweep script above consumes them. If you run the sweep and then try to check the shearing sheet, you will find the shearing sheet's files gone. Just re-run that pair to recreate them:

```
cd ~/work/rebound_rust/porttest
./problem_test 400
../target/release/examples/shearing_sheet_test 400
```

(`problem_test` is the compiled `problem_test.c`, the C side of the shearing-sheet test; the Windows edition named the same binary `rebound_test.exe`.)

Integrator	Configurations tested (all identical)
none	`none`
ias15	`ias15` (plus a separate 1000-step adaptive run)
leapfrog	orders 2, 4, 6, 8
whfast	default, `c11`, `c17`(+corrector2), `dh`, `whds`, `bary`, `mk`, `comp`, `lazy`, `usafe`
saba	default, 1, 2, 3, 4, `cm2`, `cl2`, `104`, `864`, `h844`, `h864`, `h1064`, `usafe`
janus	default (6), 2, 4, 8, 10
eos	default, all nine φ diagonals, `2-7`, `5-8`, `usafe`
mercurius	default, `usafe`, `c4`, `c5`, `inf`, `hill01` — the default keeps the moon inside the planet's critical radius, so the IAS15 close-encounter machinery runs on **every** step (also verified to 5000 steps)
bs	default, `tight` (10^-11), `loose` (10^-6), `maxdt`
trace	default, `pbs`, `ias15`, `hill1`, `perinone`, `eta001` — cross-checked to genuinely take the BS-encounter and pericentre paths

This matrix was re-run in full after every change to the crate, including after adding the `extras` field needed by REBOUNDx. It has always been 63/63.

15.2 The shearing sheet — the acceptance test

The stock Saturn's-rings example: SEI integrator, octree gravity, tree collision search, shear-periodic boundary, hard-sphere collisions with the Bridges bounce law, and `rand_r` initial conditions. Seed 42, **1,482 particles**, 400 steps, **102,478 collisions** as measured here (the Windows run counted 102,533 — same setup, that platform's maths library, hence that platform's equally valid trajectory).

```
cd ~/work/rebound_rust/porttest
./problem_test 400
../target/release/examples/shearing_sheet_test 400
shasum -a 256 state_c_final.txt state_rust_final.txt
```

Result: byte-identical, matching SHA-256

```
418c864dd1a610cbe8ea6d81ecafa1e4ce6d36837494177d9875ee820ef0766f
```

(The Windows edition's matching pair hashed to `75bdaab7...` — a *different* number than this one, and that is expected: `exp` and `log` inside the bounce law come from each platform's own maths library, so the two platforms' trajectories differ from each other while C and Rust agree exactly *within* each platform. The pass condition is, and always was, the C-vs-Rust match on the same machine.)

This test earned its keep on both platforms, by failing in an instructive way on each — two different detective stories with the same moral, that one different bit is enough to fork a chaotic run.

The Windows story: `pow`. On the original Windows port, the first attempt drifted in 330 particles. The runs stayed identical for 77 steps and separated at step 78, in one collision: the Bridges coefficient-of-restitution law computes `0.32 * pow(v, -0.234)`, and at that one impact speed Rust's own `pow` and Microsoft's differed in the last bit — one part in 10^16. That changed the bounce by one bit, which changed which tree cells the particles landed in, which changed the *set* of collisions found, and the runs diverged completely. The fix changed no physics: rewriting the identical formula as `exp(-0.234 * log(v))` on **both** sides removed the one divergent function, and the whole run became bit-identical. Section 16 tells that story in full.

The macOS story: `rand_r`. On this machine the first attempt failed *before the physics even started*: the C placed **1,441** particles where every recorded run places 1,482, because the C reference was silently using Apple's `rand_r` instead of the glibc algorithm REBOUND vendors on Windows (the `#ifdef _WIN32` guard — section 10c has the whole story). Linking the one-file `rand_r` shim restored the reference stream, and the 400-step run matched byte for byte on the first try after it. Note what did *not* need touching on macOS: the `exp/log` form of the bounce law kept working unchanged, because on this platform every maths function — `pow` included — is bit-identical between C and Rust (§15.0).

Two traps from the Windows record still worth knowing: the stock C example seeds its generator from the clock and process id (pinned to seed 42 for the comparison), and the stock example never terminates, which is why the comparison uses the `_test` variants on both sides.

15.3 Orbital derivatives — 65 functions

All 65 `reb_particle_derivative_*` functions, over two independent particle/primary configurations. **Result: 130/130 output lines bit-identical.**

15.4 Frequency analysis

A synthetic three-frequency signal (0.30, 0.55, 0.11 radians per sample), 256 samples, analysed in all three modes. **Result: MFT, FMFT and FMFT2 all bit-identical.** This exercises the FFT, the Hanning window, golden-section maximisation, Gram-Schmidt orthogonalisation and the amplitude sort end to end.

15.5 Simulationarchive — cross-language round trips

The strongest interoperability test. One implementation runs 3 × 100 steps, saving a snapshot after each 100. The *other* implementation loads snapshot 1 (the 200-step state), continues for 100 more steps, and must land on the writer's 300-step state **bit-exactly**.

```
cd ~/work/rebound_rust/porttest
./archive_test whfast-usafe write
../target/release/examples/archive_test whfast-usafe continue
../target/release/examples/archive_test whfast-usafe write
./archive_test whfast-usafe continue
```

After each `continue`, compare the continuer's dump against the writer's: `cmp archive_state_c.txt archive_state_rust.txt` staying silent is the pass. (The writer-state dumps of the two languages also match each other — that is the sixth direction in the tally.)

Result: identical in all four directions, for `whfast-usafe` (which round-trips unsynchronised Jacobi coordinates) and `ias15` (which round-trips the adaptive-step restart arrays). Archive files,

including the incremental diff-blob append format, are fully interchangeable between the C and Rust builds.

Why the `.bin` files are not committed

Run the commands above and `porttest/` fills with `archive_c_*.bin` and `archive_rust_*.bin`. Those files are **not** kept in this repository, and the reason is worth knowing, because it is a real difference between the two languages that the rest of this document does not otherwise show.

A C `struct` usually contains *padding* — unused bytes the compiler inserts so that each member starts at an address the processor likes. When C writes a whole struct to a file in one go, it writes the padding too, and nothing requires the padding to have been set to anything. It holds whatever happened to be in that memory beforehand.

Inspecting the committed `archive_c_whfast-usafe.bin` showed exactly that. Six times over, sitting between two perfectly ordinary floating-point numbers, was a fragment of text that had nothing to do with the simulation:

```
AppData\Local\pnpm;C:\Users\...\.lmstudio\bin;C:\Users\
```

That is a piece of the `PATH` environment variable of the machine that wrote the file — leftover heap memory, captured and published by accident. It is harmless here, but the same mechanism could just as easily have caught something that mattered.

The Rust port does not do this. Safe Rust has no uninitialised memory: the port builds its output buffer explicitly, so its padding bytes are zero. The `archive_rust_*.bin` files contain no such fragments — checked, and they are clean.

So the `.bin` files are treated as what they are — regenerable output — and `porttest/*.bin` is in `.gitignore`. Nothing is lost: every command needed to recreate them is printed above, and the comparison is the point, not the files.

15.6 The web server

The Rust example pauses a 100-step simulation and serves it over HTTP; the blob is then loaded by the C build.

```
cd ~/work/rebound_rust/porttest
# in one terminal:
../target/release/examples/server_test
# in another:
curl -s http://localhost:12873/simulation --output served.bin
curl -s http://localhost:12873/keyboard/81
./archive_test whfast load served.bin
cmp archive_state_c.txt server_state_rust.txt && echo identical
```

Result: the C build loads the Rust-served blob to the bit-identical state (3,448 bytes, header `REBOUND Binary File. Version: 5.`).

15.7 `add_fmt` and the built-in datasets

Built-in "solar system" dataset plus particles created from orbital elements, from Pal coordinates, and from an orbital period. **Result: all 12 particles bit-identical.**

15.8 REBOUNDx: the `tides_spin` acceptance tests

Three of REBOUNDx's own shipped examples were run in both languages and compared bit for bit. Together they exercise the tidal and spin forces, the spin differential-equation machinery, general-relativistic precession, migration forces, both a fixed-step and an **adaptive** integrator, the rotation into the invariable plane, and changing a parameter mid-run.

```
cd ~/work/reboundx_rust/porttest
./tides_spin_pseudo_c 62.83185307179586 2>/dev/null
../target/release/examples/tides_spin_pseudo 62.83185307179586
cmp state_pseudo_c.txt state_pseudo_rust.txt && echo identical
```

(The `2>/dev/null` matters: REBOUNDx prints a velocity-dependent-force warning on *every* timestep, and the long runs would otherwise write hundreds of megabytes of repeated warning text.)

Result: bit-identical in all six runs.

Test	What it exercises	End time	Result
`tides_spin_pseudo_synchronization`	WHFast, `tides_spin`, spin ODE	10 orbits	**identical**
"	"	100 orbits	**identical**
`tides_spin_kozai`	**adaptive IAS15**, `tides_spin` + `gr_potential`	t = 1,000	**identical**
"	"	t = 100,000 (full default)	**identical**
`tides_spin_migration_driven_ballistic`	WHFast, `tides_spin` + `modify_orbits_forces`, mid-run parameter change	10 orbits	**identical**
"	"	100 orbits	**identical**

Every position, velocity, mass **and spin vector** matched to all 64 bits.

The Kozai result is the strongest of the three: because IAS15 chooses its own step size, and those choices depend on the REBOUNDx forces, matching bit for bit at t = 100,000 means both programs took **the identical sequence of thousands of adaptive steps** — not merely that they arrived at the same place.

Every command needed to build the C reference — including the macOS `rand_r` shim — is given in full in section 10 and section 19 of this document. (The same three tests are also written up on their own, with a longer narrative, in `reboundx_port_test.md`; you do not need that file, because everything required to reproduce the result is here.)

15.9 REBOUNDx binary files: C writes, Rust reads, and the reverse

REBOUNDx can save a whole configuration — every force, every operator, every operator step and every parameter attached to every particle — to a binary file. If the Rust wrote that file even slightly differently, the two libraries could no longer exchange data, and a saved simulation would be trapped in whichever language created it.

So both libraries were made to build the *same* configuration and write it out: two forces (`gr_potential`, `central_force`), two operators (`modify_mass`, `drift`) across three operator steps with pre- and

post-timestep timings, and parameters of every supported type — `double`, `int` (including a negative one), `uint32`, `vec3d`, `string`, and a pointer to a force.

```
cd ~/work/reboundx_rust/porttest
../target/release/examples/rebx_binary_roundtrip     # Rust writes rebx_binary_roundtrip.bin
./rebx_binary_roundtrip_c rebx_c_reference.bin       # C writes rebx_c_reference.bin
```

Compare the two files byte by byte (`cmp -l` lists every differing byte, one per line, so counting its lines counts the differences):

```
ls -l rebx_binary_roundtrip.bin rebx_c_reference.bin
cmp -l rebx_binary_roundtrip.bin rebx_c_reference.bin | wc -l
```

Result: both files are 10,784 bytes, and 10,758 of those bytes are identical — 26 differ. (The Windows edition measured 6,392-byte files from an earlier revision of the harness configuration; the 26-byte difference, and its cause, were exactly the same there.)

The 26 bytes that differ are offsets 37–62 — a single field in the file header, and it contains no simulation data at all. It is the *git hash*: a stamp recording which exact revision of the source the library was compiled from. The C build had no git information available when it was compiled, so it writes the literal text `notavailable00000000000000`; the Rust crate is not built through a git-aware makefile at all, so it writes 26 zero bytes. This is discussed as a deliberate deviation in section 14. **Every byte of actual physics — all masses, positions, velocities, parameters, names, types and orderings — matches.**

Identical bytes are the strong result, but the practical question is whether each library can *read* what the other wrote. Both directions were tested:

```
# Rust reads the file the C wrote, then re-serializes it
../target/release/examples/rebx_binary_roundtrip rebx_c_reference.bin

# C reads the file Rust wrote, and prints everything it recovered
./rebx_binary_read_c rebx_binary_roundtrip.bin
```

Direction	Result
Rust reads the C's file	**25 / 25 checks passed**, and re-serializing reproduces all 10,784 bytes
C reads the Rust's file	all 28 lines of recovered state **identical** to the C reading its own file

That last row is worth stating plainly: the C library was asked to read the Rust-written file and dump everything it found, then asked to do the same with its own file. The two dumps are identical line for line — including raw 64-bit patterns such as `p1.tau_mass 4636b0a8e891ffff`, the negative integer `p1.primary -12345`, the three components of the `Omega` vector, the string `p1.force gr_potential`, and the *order* the parameters come back in.

To reproduce that comparison:

```
./rebx_binary_read_c rebx_binary_roundtrip.bin 2>/dev/null > readback_from_rust.txt
./rebx_binary_read_c rebx_c_reference.bin      2>/dev/null > readback_from_c.txt
cmp readback_from_rust.txt readback_from_c.txt && echo identical
```

`cmp` staying silent (and `identical` printing) means the files agree.

15.10 Three real defects the new test suite found

The Rust test suites (394 tests for REBOUND) were written from the C source rather than from the Rust, which is what made them able to find genuine translation defects. Three were found, confirmed against the C, and fixed. They are recorded here because "we tested it and found nothing" and "we tested it and found three things" are very different claims.

Defect 1 — the Kepler solver could hang forever

Symptom. A test of near-rectilinear (almost straight-line) hyperbolic motion never finished.

Diagnosis. For that motion the pericentre distance `q` is ~0, so the bisection fallback's bounds become `X_max = dt/q = +inf` and `X_min = NaN`. The C's loop is

```
} while (fastabs(X_max-X_min) > fastabs((X_max+X_min)*1e-15));
```

With NaN, `A > B` is **false**, so the C exits after one pass. The Rust had translated the exit as

```
if fastabs(X_max - X_min) <= fastabs((X_max + X_min) * 1e-15) { break; }
```

and `A <= B` is **also false** for NaN — so it never breaks. `while (A > B)` and `if (A <= B) break` are *not* equivalent in the presence of NaN.

Fix. Negate the C's condition exactly: `if !(A > B) { break; }`.

Verification. A probe calling the solver directly in both languages (`porttest/kepler_rectilinear_c.c` and `examples/kepler_rectilinear.rs`) now agrees bit for bit at h = 0 and h = 10^{-12}, and at dt = 0.1, 0.5, 2.0, 5.0 and −2.0. Before the fix, dt = 2.0 hung in Rust and returned normally in C.

Follow-up. Every other `do { } while (float condition)` in the C was then audited for the same mistranslation. One more used the fragile idiom (the Plummer-sphere rejection loop in `tools.c`); it is provably NaN-free there, but it was rewritten in the faithful `!(a > b)` form anyway.

Defect 2 — archives reported major version 0

Symptom. A test asserting that a saved archive reports REBOUND's major version failed.

Diagnosis. The header reads `"REBOUND Binary File. Version: 5.1.1"`. The C splits it at the `:` and the two `.`s, so the major-version substring is `" 5"` — **with a leading space**. C's `atoi(" 5")` skips leading whitespace and gives

1. The Rust helper collected a leading run of digits, hit the space first, and

gave 0. Minor and patch, which have no leading space, parsed correctly, which is why nothing else showed it.

Fix. Make the helper do what C's `atoi` does: skip whitespace, accept an optional sign, then take the digits.

Defect 3 — the variational centre-of-mass shift was wrong

Symptom. An audit comparing `tools.c` line by line flagged that the first-order variational `dm` accumulator summed the wrong array.

Diagnosis. The C is

```
for (size_t i=0;i<N;i++){ dm += particles[i+index].m; }   /* REAL particles */
```

and the Rust summed `particles_var` instead. `dm` multiplies a whole term of the centre-of-mass shift, so for MEGNO/Lyapunov runs (where the variational masses are zero) that term vanished entirely in Rust and did not in C. A second, related problem: the whole **second-order** variational block was untranslated, and the early `return` that stood in for it also skipped the ordinary particle shift *and* the boundary check, leaving the simulation partly transformed.

Fix. Sum the real particle array, exactly as the C does — even though that looks like an upstream slip, because bit-exactness with C 5.1.1 is the whole point. Port the full second-order block, and restore the C's **two-pass** ordering (all second-order configurations before any first-order one, because the second-order shift reads the first-order particles pre-shift).

One case genuinely cannot be reproduced: for a first-order configuration with `index > 0`, the C reads `particles[i+index]` *past the end of the particle array*. That is undefined behaviour upstream. Safe Rust cannot reproduce whatever bytes happen to follow the array, so that case reports a clear error rather than inventing a value — in keeping with the project rule to report missing or unreproducible things, never to fake them.

Verification. A Sun+Jupiter MEGNO probe (`porttest/movetocom_var_c.c` and `examples/movetocom_var.rs`) now produces bit-identical output in both languages.

After the fixes

All three fixes touch code on the bit-identity paths, so the entire verification suite was re-run from scratch afterwards: **63/63 integrator configurations identical, the shearing sheet identical (same SHA-256 as before the fixes), all three REBOUNDx acceptance tests identical, and 394 REBOUND tests passing with none ignored.**

16. The `pow` story: a Windows difference this platform does not have

`pow(a, b)` — "a raised to the power b" — deserves its own section, because on Windows it was the single maths function where the two languages disagreed, and on macOS it is not.

The macOS measurement first. On this machine, `pow` is bit-identical between the clang-built C and Rust over 200,000 general samples (`libm_diff`, §15.0) *and* over 200,000 samples shaped exactly like the BS integrator's step-size chooser (`bs_pow_diff`, below). The reason: on macOS both languages call Apple's system `pow`. There is one implementation, so there is nothing to disagree.

Reproduce that second measurement with:

```
cd ~/work/rebound_rust
cargo build --release --example bs_pow_diff
cd porttest
clang -O2 -ffp-contract=off bs_pow_diff.c -lm -o bs_pow_diff
./bs_pow_diff
../target/release/examples/bs_pow_diff
cmp bs_pow_c.txt bs_pow_rust.txt && echo "bit-identical"
```

The Windows story, kept because it teaches. On Windows, Rust ships its own `pow` implementation rather than calling Microsoft's, and the two disagreed on about **0.03% of inputs, by at most 2 ULP** (general sweep: 60 of 200,000 samples; BS-controller shapes: 56 of 200,000, every one exactly

1 ULP). Both implementations are correct to within what the C standard requires — they simply round a handful of cases differently. The Windows port caught this twice, in completely different places, and both times the diagnosis was conclusive:

1. **The shearing sheet.** The Bridges bounce law calls `pow`. One collision in step 78 came out 1 ULP different, and from there the trajectories diverged. Rewriting the identical formula as `exp(-0.234*log(x))` on *both* sides — using functions proven bit-identical there — made the whole 400-step run match exactly. That control experiment isolated `pow` as the sole cause. (The rewrite ships in both the C harness and the Rust example, which is why the macOS run in §15.2 never had to think about it.)

2. **The BS integrator.** BS chooses its own step size using `pow`. Running the three-body test on Windows: steps 1 – 2,559 everything bit-identical; step 2,560 all particle positions and velocities still bit-identical, but the *proposed next step size* differed by exactly 1 ULP (`3fce7444d03af04e` vs `3fce7444d03af04d`). Evaluating `pow` with exactly the argument shapes that chooser uses — `pow(error/0.65, 1/(2k+1))` for k = 1..8 — reproduced the same rate and magnitude: same function, same 1-ULP disagreements. The physics code was identical; only the platform `pow` differed.

What this means for you, on this Mac. Nothing needs avoiding: every maths function REBOUND calls, `pow` included, is bit-identical between the C reference and this port. If you move your simulation to a *different* platform — Windows, or a glibc-Linux machine — expect the usual chaotic- system caveat: a last-bit difference in any library function will eventually decorrelate two runs, and both remain equally valid.

17. Lint policy: why we waive some compiler suggestions

Rust has an optional extra-strict style advisor called **clippy**. Left unconfigured, it reported 469 suggestions across 23 categories on this code.

Every one of those 469 fires on a pattern that deliberately mirrors the C source, and applying clippy's suggestion would either change floating-point evaluation order (which changes results, because floating-point addition is not associative) or destroy the line-by-line correspondence to the C that makes this port reviewable at all.

So rather than either applying them or ignoring them, each category is **waived explicitly in the source**, on one line, with the reason written beside it. You will find the block at the top of `rebound_rust/src/lib.rs` and `reboundx_rust/src/lib.rs`, and repeated in each test and example file (in Rust, a test file is a separate crate and does not inherit the main crate's settings).

Clippy lint	Occurrences	Why it is waived
`excessive_precision`	274	Physical constants and Solar System data carry the C's exact digits. Truncating them would change values.
`identity_op`	56	`m[0 + 4*0]` and `y[i*6 + 0]` keep the C's row/column and stride arithmetic visible.
`erasing_op`	49	Same reason: `0 * n` appears inside index expressions that mirror the C.

Clippy lint	Occurrences	Why it is waived
`needless_range_loop`	27	`for i in 0..N { ... particles[i] ... }` mirrors the C's `for(i=0;i<N;i++)`. Iterator rewrites obscure aliasing and index relationships.
`assign_op_pattern`	10	`a = a + b` mirrors the C statement exactly.
`field_reassign_with_default`	8	Mirrors C's `struct X x = {0}; x.a = ...` initialisation idiom.
`too_many_arguments`	7	C signatures are preserved verbatim, as required.
`neg_cmp_op_on_partial_ord`	2	**Load-bearing — see the warning below.**
others (15 kinds)	36	Same category: faithful-but-unidiomatic transcription.

The one waiver you must never "clean up"

`neg_cmp_op_on_partial_ord` fires on two lines that look like they could be tidied, and tidying either of them would reintroduce a bug that made the program hang forever:

```
if !(fastabs(X_max - X_min) > fastabs((X_max + X_min) * 1e-15)) {
    break;
}
```

Clippy suggests rewriting `!(a > b)` as `a <= b`. **Those are not the same thing.** If either value is `NaN` — "not a number", which really does occur here for nearly-straight-line hyperbolic orbits — then `a > b` is false *and* `a <= b` is false. The C loop is `while (a > b)`, so with a `NaN` it exits immediately; the "tidied" Rust would never break and would spin forever. This is Defect 1 in section 15.10, and the comment in `lib.rs` says so at the point of the waiver.

The practical consequence of doing it this way:

```
cd ~/work/rebound_rust
cargo build --release          # zero warnings
cargo clippy --release --all-targets   # zero warnings
```

```
cd ~/work/reboundx_rust
cargo build --release          # zero warnings
cargo clippy --release --all-targets   # zero warnings
```

Both crates are clean under both tools, and nothing is hidden: the waivers are in the source where a reviewer reads the code, not buried in a configuration file. To see the underlying suggestions again, delete the `#![allow(clippy::...)]` block from `src/lib.rs` and re-run clippy.

18. Known limitations

1. **`pow` differs from the C on Windows only** — fully characterised in section 16. On macOS every maths function, `pow` included, is bit-identical between the C reference and this port.
2. **WHFast512 does not integrate on Windows or on Apple Silicon** — in C *or* Rust. All four produce the identical "AVX512 is not supported on your platform." error, because the fast core is x86-only AVX-512 assembly: MSVC does not compile it, and an arm64 processor cannot run it.

3. **Excluded subsystems**: the OpenGL 3-D display (`display.c`, `glad.c`) and MPI (`communication_mpi.c`). Neither is part of either reference C build (section 9). The browser-based viewer, which *is* the ported visualisation path, works the same on both platforms.

4. **Not carried, and why**: - `reb_simulation_output_screenshot` — needs the browser display round-trip; - `reb_integrator_register` (registering your own integrator at run time) — the Rust integrator set is a closed `enum`; - the WHFast512 AVX-512 assembly core.

5. **The C's own documented restrictions carry over unchanged** — for example MERCURIUS and TRACE emit the same warnings about variational equations and collision-search modes.

6. **`cargo clippy` is not clean** by design; see section 17.

19. How to reproduce every result yourself

In order, from a fresh Mac (with the two installs from section 4). If you cloned the `rustSolveIt_macos-silicon_SUND` repository, its root is your `~/work` and steps 1's first line is already done for the Rust crates — you only add the two C reference trees beside them.

```
# 1. Get the source
cd ~/work
git clone https://github.com/hannorein/rebound.git rebound/rebound
(cd rebound/rebound && git checkout dad5f97806ecbb408dcaff728851c64e67f9f6eb)
git clone https://github.com/dtamayo/reboundx.git reboundx

# 2. Build the C REBOUND reference (section 10a explains the flags)
cd ~/work/rebound/rebound/src
clang -c -DBUILDINGLIBREBOUND -D_GNU_SOURCE -DSERVER \
      -DGITHASH=dad5f97806ecbb408dcaff728851c64e67f9f6eb \
      -O2 -ffp-contract=off *.c
ar rcs librebound_static.a *.o

# 3. Build the C REBOUNDx reference (no shim needed under clang -- section 10b)
cd ~/work/reboundx/src
clang -c -I../../rebound/rebound/src -I. -D_GNU_SOURCE -DLIBREBOUNDX \
      -O2 -ffp-contract=off *.c
ar rcs libreboundx.a *.o

# 4. Build the Rust crates and run their test suites (394 + 137 tests)
cd ~/work/rebound_rust
cargo build --release
cargo test --release
cd ~/work/reboundx_rust
cargo build --release
cargo test --release
```

Step 5 — build the C comparison harnesses

A "harness" is a small C program that builds one specific experiment and prints every resulting number as raw bits, so it can be compared with the Rust twin of the same name. There are eleven for REBOUND and five for REBOUNDx.

First compile the macOS `rand_r` shim once (section 10c explains why):

```
cd ~/work/rebound_rust/porttest
clang -c -O2 -ffp-contract=off macos_shim/rand_r_glibc.c -o macos_shim/rand_r_glibc.o
```

Build the eleven REBOUND harnesses, each linked with the shim and the static library:

```
cd ~/work/rebound_rust/porttest
for f in addfmt_test archive_test bs_pow_diff derivatives_test \
         frequency_test integrators_test kepler_rectilinear_c libm_diff \
         movetocom_var_c movetocom_var_test problem_test; do
  clang -I../../rebound/rebound/src -D_GNU_SOURCE -O2 -ffp-contract=off \
        "$f.c" macos_shim/rand_r_glibc.o \
        ../../rebound/rebound/src/librebound_static.a -lm -o "$f"
done
```

(`libm_diff` and `bs_pow_diff` never call REBOUND, so for those two the shim and library are simply unused; linking them anyway keeps the loop uniform. Two harnesses print a benign compiler warning each — a `%zu` format nit and a pointer-cast nit, both in the *harness* code, not the libraries.)

Build the five REBOUNDx harnesses (note the link order — the REBOUNDx library first, then REBOUND's):

```
cd ~/work/reboundx_rust/porttest
for f in tides_spin_pseudo_c tides_spin_kozai_c tides_spin_migration_c \
         rebx_binary_roundtrip_c rebx_binary_read_c; do
  clang -I../../rebound/rebound/src -I../../reboundx/src -D_GNU_SOURCE \
        -O2 -ffp-contract=off "$f.c" \
        ../../rebound_rust/porttest/macos_shim/rand_r_glibc.o \
        ../../reboundx/src/libreboundx.a \
        ../../rebound/rebound/src/librebound_static.a -lm -o "$f"
done
```

Step 6 — run each matched pair and compare

Each harness has a Rust twin with the same name under `examples/`. Run both, then compare. The comparison always works the same way: `cmp` staying silent means the files are identical.

The integrator matrix, all 63 configurations in one command:

```
cd ~/work/rebound_rust/porttest
bash run_integrator_matrix.sh 500
```

It must end `ALL CONFIGURATIONS BIT-IDENTICAL`.

The shearing sheet is the largest single check. Note that the Rust example to run is `shearing_sheet_test`, not `shearing_sheet`: the stock REBOUND example integrates forever by design, so the `_test` variant is the same simulation with a stopping time added.

```
cd ~/work/rebound_rust/porttest
./problem_test 400
../target/release/examples/shearing_sheet_test 400
shasum -a 256 state_c_final.txt state_rust_final.txt
```

Both hashes must read `418c864dd1a610cbe8ea6d81ecafa1e4ce6d36837494177d9875ee820ef0766f`.

The remaining REBOUND pairs follow the same pattern; each writes its own pair of dump files, named in section 15:

```
cd ~/work/rebound_rust/porttest
./libm_diff            && ../target/release/examples/libm_diff
cmp libm_c.txt libm_rust.txt && echo "libm identical"
./bs_pow_diff          && ../target/release/examples/bs_pow_diff
cmp bs_pow_c.txt bs_pow_rust.txt && echo "bs_pow identical"
./derivatives_test     && ../target/release/examples/derivatives_test
cmp derivatives_c.txt derivatives_rust.txt && echo "derivatives identical"
./frequency_test       && ../target/release/examples/frequency_test
cmp frequency_c.txt frequency_rust.txt && echo "frequency identical"
./addfmt_test          && ../target/release/examples/addfmt_test
cmp addfmt_c.txt addfmt_rust.txt && echo "addfmt identical"
./movetocom_var_c      && ../target/release/examples/movetocom_var
cmp movetocom_var_c.txt movetocom_var_rust.txt && echo "movetocom identical"
```

The Simulationarchive round trips and the web server test are run with the commands printed in sections 15.5 and 15.6.

The three REBOUNDx tidal-spin tests take a stopping time as their argument. Send the C program's error stream to `/dev/null`, because REBOUNDx prints a warning on *every* timestep and the long runs would otherwise produce hundreds of megabytes of text:

```
cd ~/work/reboundx_rust/porttest
for pair in "pseudo 62.83185307179586" "kozai 1000" "migration 62.83185307179586"; do
  n="${pair% *}"; t="${pair#* }"
  ./tides_spin_${n}_c "$t" >/dev/null 2>&1
  ../target/release/examples/tides_spin_$n "$t" >/dev/null 2>&1
  cmp -s state_${n}_c.txt state_${n}_rust.txt \
    && echo "$n : BIT-IDENTICAL" || echo "$n : MISMATCH"
done
```

That must print `BIT-IDENTICAL` three times. Run the same loop again with `628.3185307179586` for pseudo and migration and with no argument for kozai (its full default, t = 100,000) to reproduce the three long runs. The binary-file checks of §15.9 are run with the commands given in that section.

Every command in this document was actually run on the machine described in section 9, and the outputs quoted are the outputs it produced.

20. Credit, citation and licensing

The originals

REBOUND is © Hanno Rein, Shangfei Liu, Dan Tamayo, David S. Spiegel, Tiger Lu, Pejvak Javaheri, Rishit Dagli, Dave O'Hallaron, Ernst Hairer and the REBOUND contributors.

REBOUNDx is © Dan Tamayo, Hanno Rein and the REBOUNDx contributors.

Both are GPL-3.0-or-later. These translations are derivative works under the same license. Every module header names the C file it translates and its copyright holders.

How to cite

From the REBOUND project:

> If you use this code or parts of this code for results presented in a scientific publication, we would greatly appreciate a citation. The simplest way to find the citations relevant to the

> specific setup of your REBOUND simulation is: "`python sim = rebound.Simulation() # -your setup- sim.cite()` "

`sim.cite()` is part of the upstream **Python** package (`pip install rebound`), which this Rust port does not reimplement. Use the per-module citation tables in `README.md` instead, which map each feature to the papers `sim.cite()` would name.

At minimum:

- REBOUND — **Rein & Liu 2012**, *Astronomy & Astrophysics* **537**, A128.
- REBOUNDx — **Tamayo, Rein, Shi & Hernandez 2019**, *MNRAS* **491**, 2885 (arXiv:1908.05634).

Plus the paper for the integrator and for each REBOUNDx effect you switch on.

A note on the upstream projects' AI policy

REBOUND's README states:

> REBOUND is a labour of love, created by people. Please refrain from submitting issues or pull requests that have been generated by an LLM or other fully-automated tools. [...] You may of course use AI assistants for your own work with REBOUND. Just don't submit any AI generated code.

This translation was produced with AI assistance. That is **exactly the "your own work" case the policy permits**: it is a separate derivative work living in its own repository. Accordingly **nothing from this port has been or will be submitted upstream** — no issues, no pull requests. If you find a problem here, report it here, and please verify against the original C before raising anything with the upstream maintainers, who did not write this and should not be asked to support it.

Versions translated

Project	Version	Commit
REBOUND	5.1.1	`dad5f97806ecbb408dcaff728851c64e67f9f6eb`
REBOUNDx	5.1.0	latest at time of porting